

GTX 1050 Mobile

Passthrough Optimus su Proxmox

Relazione tecnica: diagnosi, recupero della VBIOS OEM, SSDT ACPI dinamica, switch idempotente tra VM Linux, Secure Boot e benchmark.

Componente	Esito
Host	Proxmox VE 9.1.5, kernel 6.17.9, IOMMU e VFIO attivi
GPU	NVIDIA GP107M GTX 1050 Mobile, PCI ID 10de:1c8d
Firmware	VBIOS OEM HP, 169472 byte, 86.07.5F.00.2C
Ubuntu	Driver NVIDIA 580.173.02, 4096 MiB, nvidia-smi valido
Kali	Driver NVIDIA installabile, SeaBIOS/Q35 mantenuto
Prova rendering	glxgears sul display NVIDIA :2, circa 24-25 mila FPS

Repository: proxmox-gtx1050-mobile-passthrough

1. Problema e criterio di successo

Una GPU mobile Optimus non è una GPU desktop isolata: il display fisico resta normalmente collegato alla iGPU e il driver NVIDIA può richiedere la propria VBIOS attraverso ACPI. Il passthrough standard con hostpci e una ROM PCI non era sufficiente.

Il criterio di successo non è solo vedere una riga in lspci. La GPU deve essere assegnabile a una VM, il driver proprietario deve caricare, nvidia-smi deve riportare nome, driver e memoria, e un processo grafico deve usare realmente la GPU.

2. Termini fondamentali

BDF	Indirizzo PCI dominio:bus:device.funzione. La GPU host è 0000:02:00.0.
VBIOS / ROM	Firmware della GPU; in questo caso il file .rom contiene la VBIOS OEM.
IOMMU e VFIO	Isolano DMA e assegnano il dispositivo reale a QEMU invece che al driver host.
ACPI	Descrizione firmware dell'hardware e dei metodi richiamati dal sistema operativo.
ASL / AML / SSDT	ASL è testo, AML è bytecode compilato, SSDT è tabella ACPI aggiuntiva.
_ROM	Metodo ACPI che fornisce una porzione della ROM al driver, dati offset e dimensione.
fw_cfg	Canale QEMU per fornire piccoli blob al guest; qui trasporta la VBIOS.
MOK	Chiave da registrare in una schermata UEFI pre-avvio per fidare moduli DKMS con Secure Boot.

3. Perché la soluzione ACPI era necessaria

Il solo romfile espone una ROM nella configurazione PCI virtuale. Il driver della GTX 1050 Mobile, nel percorso Optimus, cerca invece _ROM nel device ACPI. Per questo rombar=1 mostrava byte ROM ma non risolveva l'inizializzazione del driver.

La soluzione passa lo stesso file VBIOS OEM mediante fw_cfg. Una SSDT in AML si aggancia al device della GPU, legge il file una volta, lo mantiene in un buffer e implementa _ROM restituendo il segmento richiesto. rombar=0 resta intenzionale.

```
QEMU fw_cfg -> SSDT AML -> metodo ACPI _ROM -> driver NVIDIA
file VBIOS OEM      cache buffer      slice offset/length
```

4. PCI -> ACPI senza valori fissi

Il guest restituisce la catena PCI con lspci -PP. Ogni hop viene convertito nel nome ACPI Sxx: valore = slot * 8 + funzione.

```
00:1c.0/01:00.0
1c.0 -> 0x1c * 8 + 0 = 0xe0 -> SE0
00.0 -> 0x00 * 8 + 0 = 0x00 -> S00
Scope finale: \_SB.PCI0.SE0.S00
```

Lo script avvia brevemente la VM, scopre questa topologia reale e genera la SSDT specifica della VM. Così il metodo può adattarsi a un'altra NVIDIA mobile, se vengono indicati BDF host e VBIOS OEM corretti.

5. Switch GPU idempotente

- Menu numerato delle VM oppure --vm VMID per automazione.
- Ricerca proprietario corrente della GPU e arresto pulito della sola VM coinvolta.
- Cleanup mirato: hostpci, romfile, rombar, argomenti SSDT/fw_cfg e CPU hidden generati dallo script.
- Riavvio automatico della VM sorgente che prima era accesa.
- Discovery ACPI, generazione AML, avvio finale e verifica nvidia-smi.
- Nessuna conversione forzata di BIOS, OVMF, SeaBIOS, Q35 o vga: questi restano proprietà della VM.
- Se hostpci, SSDT/fw_cfg e nvidia-smi sono già corretti nella VM richiesta, non opera né riavvia.

Comandi principali

```
gpu-vm-switch
gpu-vm-switch --vm 1001 --yes
gpu-vm-switch --vm 1001 --dry-run --yes
gpu-vm-switch --gpu 0000:03:00 --rom /usr/share/kvm/oem.rom --vm 123 --yes
gpu-vm-switch --vm 123 --skip-drivers --yes
```

Driver automatici: Ubuntu, Debian, Kali, Arch, Fedora, RHEL, Rocky e AlmaLinux. Per Debian la sorgente contrib non-free non-free-firmware è aggiunta solo se non è già presente; la logica gestisce anche il formato Deb822.

6. Secure Boot senza automazione fittizia

La schermata MOK e fuori dall'OS: SSH e QEMU Guest Agent non possono premere tasti prima dell'avvio. Un finto disable temporaneo non risolve il problema: riattivando Secure Boot, un modulo DKMS non firmato verrebbe nuovamente rifiutato.

Lo script rileva lo stato reale nel guest, con mokutil oppure con la variabile EFI SecureBoot. In modo interattivo propone di disabilitarlo; con --yes serve sempre il consenso esplicito seguente:

```
gpu-vm-switch --vm 123 --disable-secure-boot --yes
```

L'operazione è permanente e solo OVMF/efidisk0 4m. Prima crea EFI/BOOT/BOOTX64.EFI, copia le vecchie variabili EFI in /usr/share/kvm/optimus-gpu-switch/efi-backups/, conserva il vecchio volume come unused disk e crea nuove variabili senza chiavi. Questo evita MOK ma abbassa la protezione della catena di boot.

7. Benchmark e prova di rendering

glmark2 da SSH ha fallito con Could not initialize canvas: e previsto, perche glmark2 X11 necessita un display. Il DRM della GPU mobile render-only non offre un CRTC utile al test. E stato quindi creato un Xorg NVIDIA headless sul display :2 e installato il wrapper nvidia-glxgears.

```
nvidia-glxgears
```

```
120907 frames in 5.0 seconds = 24181.367 FPS
```

```
125006 frames in 5.0 seconds = 25001.096 FPS
```

```
watch -n 1 nvidia-smi
```

```
nvidia-smi
```

htop non visualizza i contatori NVIDIA perche legge CPU e RAM del sistema operativo. nvidia-smi e nvidia-smi interrogano invece driver e GPU.

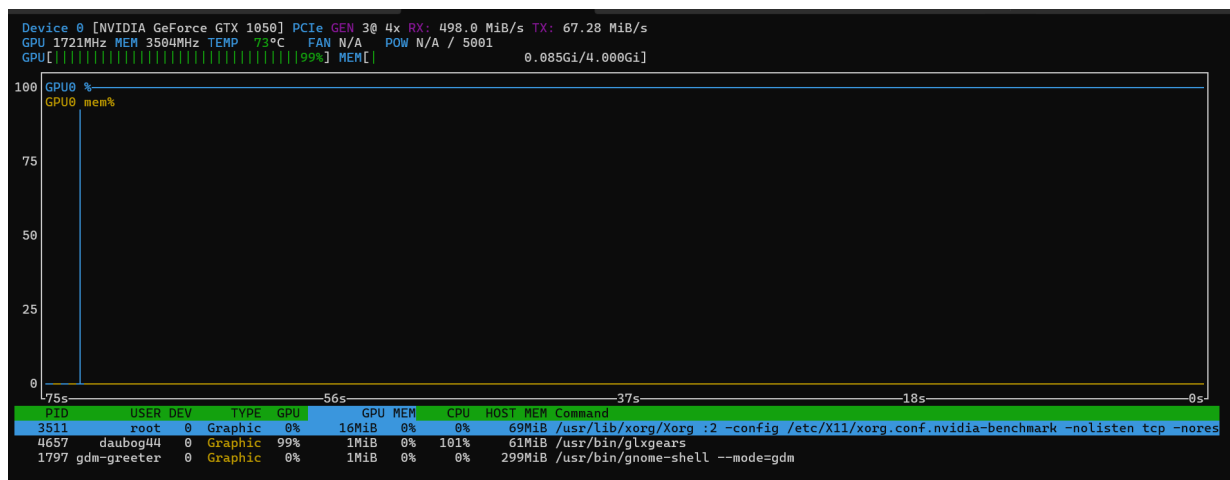


Figura 1 - nvidia-smi: glxgears usa il 99% della GPU; Xorg gira su :2.

8. Estrazione della VBIOS OEM

Lo script Python incluso cerca la signature PCI option ROM 55 aa, verifica PCIR, vendor NVIDIA e device 1c8d. Scansiona il payload RAW e stream LZMA, quindi estrae tutte le immagini della ROM fino al flag finale. E la versione ordinata e validata dello script di recupero fornito nei tentativi.

```
python3 scripts/extract_gtx1050_rom.py /tmp/084C0.bin \  
+ --device-id 1c8d \  
+ --output /usr/share/kvm/gtx1050_hp_native.rom
```

Non distribuire la ROM nel repository: e firmware OEM. Conservare il file sul nodo, usare --force solo con verifica di vendor, device ID e origine del payload, e provare l'altro payload HP se non si trova il device.

9. Tentativi che non hanno risolto il problema

Tentativo	Perche falliva	Correzione
Solo romfile / rombar	Il driver Optimus non leggeva la ROM dalla sola finestra PCI.	SSDT _ROM + fw_cfg.
ACPI scritto a mano	Cambiando bridge cambia lo scope del device.	Discovery lspci -PP e conversione Sxx.
glmark2 via SSH	Mancava un canvas X11.	Xorg NVIDIA :2 e glxgears.
Prima installazione Kali	Headers/DKMS non pronti.	Headers, build tools, DKMS e riavvio.
Repo Debian riscritta	Automazione non idempotente.	Verifica componenti prima di aggiungere.

10. Ripetere il procedimento

- Controllare che IOMMU/VFIO isolino GPU e funzione audio sul nodo.
- Estrarre la VBIOS OEM del nuovo dispositivo e verificare BDF e PCI ID con lspci.
- Configurare la VM con Q35 e Guest Agent; mantenere il firmware scelto dalla VM.
- Eseguire gpu-vm-switch con --gpu e --rom, poi nvidia-smi.
- Per un test grafico su guest muxless, creare un display X NVIDIA headless e usare nvidia-glxgears.
- Scegliere consapevolmente MOK o --disable-secure-boot per OVMF; non alternare automaticamente le due politiche.

Riferimenti

QEMU fw_cfg: https://qemu-project.gitlab.io/qemu/specs/fw_cfg.html

ACPI: <https://uefi.org/acpi/specs>

Proxmox VE Administration Guide: <https://pve.proxmox.com/pve-docs/pve-admin-guide.html>

NVIDIA driver kernel modules: <https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/kernel-modules.html>